Pilas Estáticas en Python

Estructuras de datos LIFO con capacidad fija • Implementación, usos y prácticas

Objetivos de aprendizaje

Comprender LIFO y TAD

Entender el principio "Last In, First Out" y la diferencia entre el Tipo Abstracto de Datos y su implementación.

Estática vs Dinámica

Distinguir entre implementaciones de tamaño fijo (arrays) y dinámicas (listas) en Python.

Implementar StaticStack

Construir una clase completa en Python con manejo de capacidad fija y puntero `top`.

Dominar Operaciones

Aplicar correctamente `push`, `pop`, `peek`, `is_empty` e `is_full` con complejidad O(1).

Manejo de Errores

Detectar y gestionar situaciones críticas como *Stack Overflow* (desbordamiento) y *Underflow*.

Resolver Problemas

Aplicar pilas en casos reales como validación de paréntesis balanceados y reversión de cadenas.

Estructuras de Datos Lineales

¿Qué es una Pila (Stack)?

LIFO

Last In, First Out

Definición

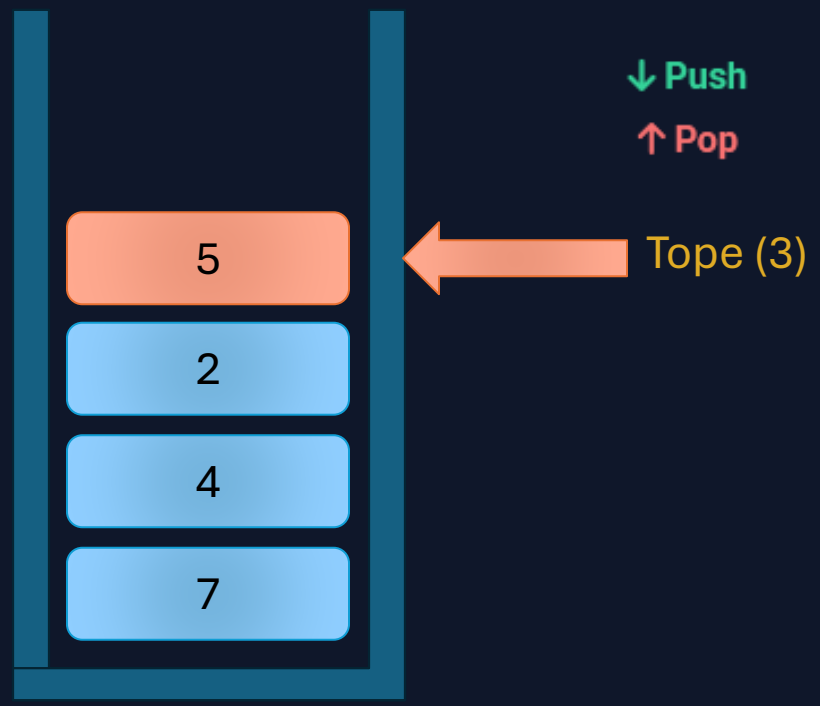
Una colección ordenada de elementos donde las inserciones y eliminaciones se realizan por un solo extremo llamad Tope

Operaciones principales

push(x) / apilar
Insertar un elemento en el tope

pop() / desapilar
Elimina y devuelve el elemento del tope

peek() / cima
Observa el elemento del tope sin eliminarlo



Índice: 0 (base) .... N(tope)

CASOS DE USO: Deshacer (Ctrl+Z) | Backtracking | Expresiones

¿Qué es una pila?

Conceptos fundamentales de la estructura LIFO

Estructura LIFO

Last In, First Out: El último elemento en entrar es el primero en salir. Similar a una pila de libros o platos.

Acceso Restringido

Solo se puede acceder, insertar o eliminar elementos desde un único extremo llamado **Tope (Top)**.

Operaciones Básicas

Todas las acciones principales (apilar/desapilar) ocurren exclusivamente sobre el elemento superior.

Usos Comunes

Parsers y Compiladores

Verificación de sintaxis y evaluación de expresiones matemáticas.

Deshacer / Rehacer

Funcionalidad "Undo" en editores de texto y software de diseño.

Call Stack

Gestión de llamadas a funciones y recursividad en la ejecución de programas.

Analogía: Pila de Platos

Concepto LIFO (Last In, First Out) en el mundo real

"El último plato que pones arriba es el primero que quitas."

PUSH (Apilar) `top += 1`

Colocar un nuevo elemento en la cima de la pila. Solo es posible si la pila no está llena (overflow).

POP (Desapilar) `return data[top--]`

Quitar y devolver el elemento que está en la cima. Requiere que la pila no esté vacía (underflow).

PEEK (Cima/Tope) `return data[top]`

Consultar el valor del elemento superior sin eliminarlo. Útil para verificar antes de desapilar.

Pila como TAD vs Implementación

Distinción conceptual clave en estructuras de datos

TAD Pila
Tipo Abstracto de Datos

Define QUÉ hace
Especifica el comportamiento lógico y las operaciones disponibles sin preocuparse por los detalles internos.

CONTRATO / INTERFAZ

- push(x)
- pop()
- peek()
- is_empty()

Implementación
Estructura concreta

Define CÓMO se construye
Determina la organización de la memoria, el rendimiento real y los algoritmos específicos utilizados.

OPCIONES COMUNES

- Array / Lista Estática. Tamaño fijo, memoria contigua.
- Lista dinámica. Redimensionable.
- Lista Enlazada. Nodos dispersos en memoria.

Componentes de una pila estática

Estructura interna y variables de control

Capacidad Fija (capacity)	Define el tamaño máximo inmutable de la pila. Es crucial para el control de memoria y la detección de desbordamientos.
Contenedor de Datos	Un array o lista preasignada con el tamaño de la **capacity**. Almacena los elementos apilados.
Índice Top	Puntero que indica la posición del último elemento insertado. Se inicializa en **-1** para indicar que la pila está vacía.
Invariante de Estado	Regla fundamental: los elementos válidos siempre ocupan las posiciones desde el índice **0** hasta **top** inclusive.

Casos de Uso Comunes

Aplicaciones prácticas de las pilas en el desarrollo de software

Deshacer / Rehacer

Funcionalidad esencial en editores de texto y gráficos. Cada acción se apila; "deshacer" elimina la última acción (pop) y la mueve a una pila de "rehacer".

Evaluación de Expresiones

Conversión y cálculo de notación infija (A+B) a postfija (AB+) o prefija (+AB). Los compiladores usan pilas para respetar la precedencia de operadores.

Verificación de Paréntesis

Validación de sintaxis en código fuente. Asegura que cada símbolo de apertura ([{ tenga su correspondiente cierre }]) en el orden correcto.

Recorrido DFS

Búsqueda en Profundidad (Depth-First Search) en grafos y árboles. Utiliza una pila para recordar los nodos visitados y retroceder (backtracking).

Pila de Llamadas (Call Stack)

Gestión fundamental de la ejecución de programas. Almacena información sobre las subrutinas activas: direcciones de retorno, parámetros y variables locales. Es el corazón de la recursividad.

Momento de Reflexión

Discusión en parejas (2 minutos)

01

¿Por qué LIFO es útil para deshacer acciones?

PISTA

Piensa en cómo escribes un documento. Si cometes un error, ¿cuál es la acción más reciente que quieres eliminar? ¿La primera que hiciste o la última?

02

¿Qué riesgos añade una capacidad fija?

PISTA

Imagina que tienes una caja donde solo caben 5 libros. ¿Qué sucede si intentas guardar el sexto libro? ¿Cómo afecta esto a la fiabilidad del programa?

Operaciones de pila

Visión general de los métodos fundamentales (TAD)

MANIPULACIÓN DE DATOS

`push(x)` Escritura

Añade el elemento **x** en la cima (top) de la pila.

`pop()` Lectura/Eliminación

Elimina y devuelve el elemento que está en la cima.

`peek()` Solo Lectura

Devuelve el elemento de la cima sin eliminarlo (inspección).

VERIFICACIÓN DE ESTADO

`is_empty()`

Comprueba si la pila no tiene elementos. Esencial para evitar errores de *Underflow* antes de hacer pop.

`is_full()`

Comprueba si la pila ha alcanzado su capacidad máxima. Crítico en **pilas estáticas** para evitar *Overflow*.

⚡ Todas estas operaciones tienen complejidad temporal **O(1)**.

PUSH en detalle

Algoritmo y visualización de la inserción paso a paso

Secuencia Lógica

1. **Verificar Estado**
   Consultar `is_full()`. Si la pila está llena, lanzar error (Overflow) para evitar sobrescribir memoria no válida.

2. **Mover Puntero**
   Incrementar el índice `top` en 1. Ahora apunta a la siguiente celda libre.

```
self.top += 1
```

3. **Insertar Dato**
   Almacenar el nuevo elemento en la posición del array señalada por `top`.

```
self._data[self.top] = item
```

Ejemplo Visual

Capacidad = 3

Atención: Si intentamos un cuarto `push('D')`, la condición `is_full()` detendrá la operación.

POP y PEEK en detalle

Operaciones de extracción y consulta en una pila estática

Operación POP

Extrae y elimina el elemento en el tope de la pila.

1. Verificar estado

Comprobar `is_empty()`. Si es verdadero, lanzar error (Underflow).

2. Recuperar valor

Guardar temporalmente el valor: `temp = data[top]`

3. Limpiar referencia

Asignar `data[top] = None` para liberar memoria y evitar referencias fantasma.

4. Decrementar puntero

Reducir el índice: `top -= 1`

5. Retornar

Devolver el valor guardado en `temp`.

Operación PEEK

Consulta el elemento en el tope sin modificar la pila.

1. Verificar estado

Igual que en POP, verificar `is_empty()` para evitar accesos inválidos.

2. Acceder y Retornar

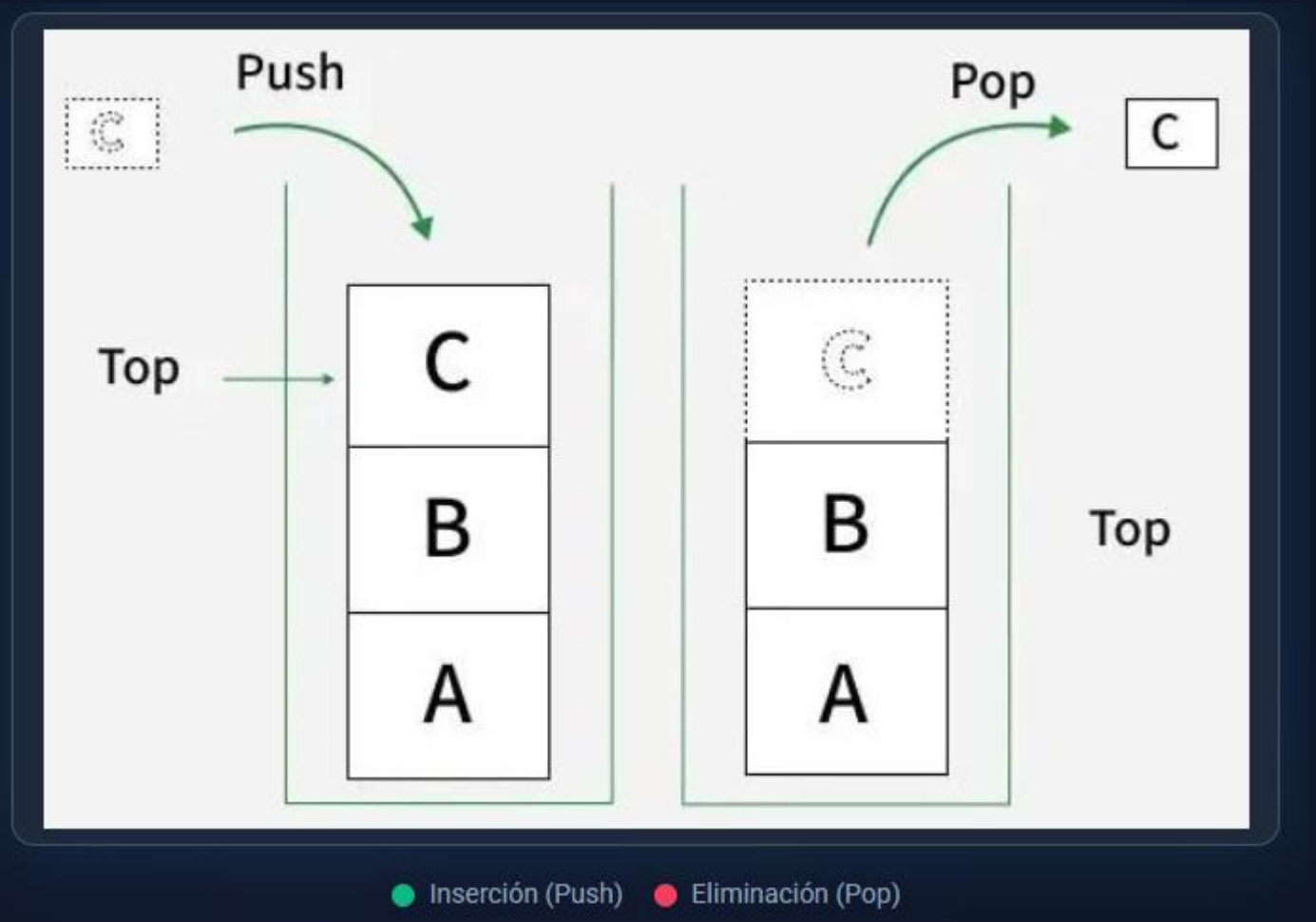
Simplemente devolver `data[top]`.

DIFERENCIA CLAVE

POP	PEEK
Modifica `top`	No modifica `top`
O(1)	O(1)

Diagrama: Operaciones PUSH y POP

Visualización del flujo de datos y movimiento del puntero 'top'



PUSH (Insertar)

`top = top + 1`

1. Verificar si hay espacio (`is_full`).
2. Incrementar el puntero `top` para apuntar a la nueva posición vacía.
3. Asignar el valor en `data[top]`.

POP (Eliminar)

`top = top - 1`

1. Verificar que no esté vacía (`is_empty`).
2. Acceder al elemento en `data[top]`.
3. Decrementar `top` para "olvidar" el dato superior.

Errores: Underflow y Overflow

Condiciones críticas en el manejo de pilas estáticas

Stack Underflow

Ocurre al intentar extraer (`pop`) o leer (`peek`) de una pila vacía. Implica acceder a índices inválidos cuando el puntero `top` es -1.

Stack Overflow

Sucede al intentar insertar (`push`) en una pila llena. No hay espacio físico en el array preasignado (`top == capacity - 1`).

ESTRATEGIAS DE PREVENCIÓN

Validación Previa

Es obligatorio verificar el estado con `is_empty()` antes de extraer y `is_full()` antes de insertar para evitar fallos de memoria.

Excepciones Claras

En lugar de fallar silenciosamente, lanzar errores descriptivos (e.g., `IndexError`, `OverflowError`) facilita enormemente la depuración.

Traza de Ejecución Manual

Simula las operaciones en papel

Escenario

1. Tienes una pila vacía inicialmente.
2. La capacidad máxima es **3 elementos** (`cap = 3`).

Secuencia de Operaciones:

```
1. push(1)
2. push(2)
3. peek()
4. push(3)
5. push(4)
6. pop()
```

Preguntas a Resolver

Pregunta A

¿En qué paso exacto ocurre un error de **Overflow** (desbordamiento)?

Pregunta B

¿Cuál es el valor del elemento en el **top** (cima) después de ejecutar el último `pop()`?

Pseudocódigo: StaticStack

Lógica algorítmica independiente del lenguaje

Estado Inicial

La pila comienza vacía con `top = -1`. El array tiene un tamaño fijo definido por `capacity`.

↓ PUSH (Insertar)

Primero verificamos si cabe (`is_full`). Luego incrementamos el puntero y guardamos el dato.

↑ POP (Extraer)

Verificamos si hay algo (`is_empty`). Recuperamos el dato, limpiamos la celda y decrementamos el puntero.

Clase StaticStack: Esqueleto

Implementación básica de la estructura y constructor

static_stack.py

```python
class StaticStack:
    """Implementación de pila usando lista de tamaño fijo."""

    def __init__(self, capacity):
        # Validar que la capacidad sea positiva
        if capacity <= 0:
            raise ValueError("La capacidad debe ser > 0")

        self.capacity = capacity

        # Preasignar espacio con None (simula array fijo)
        self._data = [None] * capacity

        # Inicializar puntero top (vacío = -1)
        self.top = -1
```

Validación de Entrada

Es crucial asegurar que `capacity` sea un número positivo. Una pila con tamaño 0 o negativo no tiene sentido lógico y causaría errores inmediatos.

Preasignación de Memoria

`[None] * capacity` crea una lista con el tamaño exacto desde el inicio. Esto simula el comportamiento de un array estático en C/Java, evitando redimensionamientos automáticos.

Puntero Top

Inicializamos `self.top = -1` para indicar que la pila está vacía. El primer elemento (índice 0) se insertará al incrementar top a 0.

Método push(x)

Inserción de elementos, validación de estado y manejo de errores

static_stack.py

```python
def is_full(self):
    # Verifica si el puntero llegó al límite
    return self.top + 1 == self.capacity

def push(self, item):
    # 1. Protección contra Overflow
    if self.is_full():
        raise OverflowError("Stack overflow: Pila llena")

    # 2. Mover puntero y luego asignar valor
    self.top += 1
    self._data[self.top] = item
```

⚠ Control de Overflow

Antes de cualquier inserción, **es obligatorio** verificar si la pila está llena. Si `is_full()` es True, detenemos la ejecución lanzando un error para proteger la memoria.

↑ Incremento del Puntero

La clave lógica: primero incrementamos `top` para apuntar a la siguiente celda vacía, y *después* escribimos el dato.

`Inicio: top=-1 -> push -> top=0 (dato en índice 0)`

⚡ Complejidad O(1)

No hay bucles ni desplazamientos de memoria. Independientemente del tamaño de la pila, la operación `push` toma un tiempo constante.

Métodos pop() y peek()

Extracción y consulta de elementos con manejo de errores (Underflow)

static_stack.py

```python
def is_empty(self):
    return self.top == -1

def pop(self):
    if self.is_empty():
        raise IndexError("Stack underflow")

    item = self._data[self.top]
    self._data[self.top] = None # Limpiar ref
    self.top -= 1
    return item

def peek(self):
    if self.is_empty():
        raise IndexError("Stack vacía")
    return self._data[self.top]
```

⚠ Protección Underflow

Antes de extraer (pop) o mirar (peek), es **obligatorio** verificar si la pila está vacía (`is_empty`). Si lo está, lanzamos un `IndexError`.

🧹 Limpieza de Memoria

`_data[top] = None` es vital en Python. Elimina la referencia al objeto extraído, permitiendo que el *Garbage Collector* libere esa memoria si ya no se usa.

👁 Diferencia Peek vs Pop

`peek()` solo **lee** el valor sin modificar `top`. `pop()` **extrae** el valor y decrementa `top` para "olvidar" la posición.

💡 *Tip de Depuración: Asignar `None` también ayuda visualmente al imprimir el array interno `_data`, mostrando claramente qué celdas están "libres".*

Método size()

Calculando el tamaño actual sin iteraciones

static_stack.py

```python
def size(self):
    """Devuelve el número de elementos en la pila."""
    # Operación matemática simple: O(1)
    return self.top + 1

# --- Ejemplos de uso ---
def demo_size():
    pila = StaticStack(5)
    print(pila.size()) # Output: 0 (Vacía)

    pila.push('A')
    pila.push('B')
    print(pila.size()) # Output: 2 (Medio llena)

    # ... insertar 3 más ...
    print(pila.size()) # Output: 5 (Llena)
```

Dado que los índices comienzan en 0, la cantidad de elementos es simplemente el índice del tope más uno.

`size = top + 1`

Complejidad Constante O(1)

No necesitamos recorrer el array ni usar contadores en un bucle. El cálculo es instantáneo, sin importar si la pila tiene 10 o 1 millón de elementos.

Escenarios Visuales

VACÍA top = -1	→	size = 0
PARCIAL top = 1	→	size = 2
LLENA top = 4	→	size = 5

Implementación con Array Preasignado

Estructura interna: Índices, Top y Capacidad Fija

Datos Activos (0..top)

Los elementos válidos de la pila residen en los índices desde `0` hasta `top`. Estos son los datos accesibles mediante operaciones *pop* o *peek*.

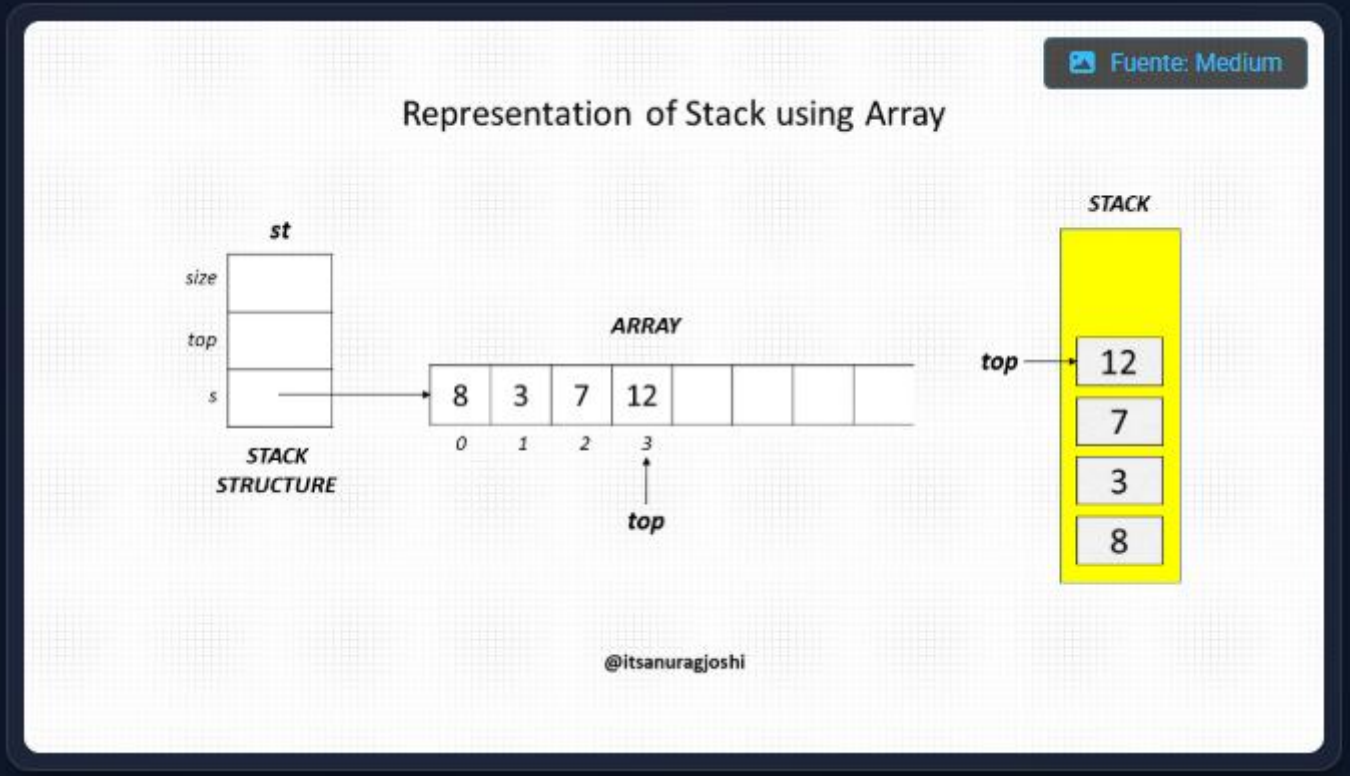
Espacio Libre (None)

Desde `top + 1` hasta `capacity - 1`. Estas celdas están reservadas en memoria (preasignadas) pero se consideran "vacías". Usualmente se llenan con `None` al hacer *pop*.

Puntero Top

Es la variable crítica que define el estado de la pila.

- `top == -1 : Pila vacía`
- `top == capacity - 1 : Pila llena (Overflow)`



Revertir Cadena & Aplicaciones

Uso clásico de LIFO para inversión y análisis sintáctico

string_utils.py

```python
def reverse_string(text):
    # 1. Crear pila con tamaño exacto (sin overflow)
    stack = StaticStack(len(text))

    # 2. Apilar cada caracter (Orden Natural)
    for char in text:
        stack.push(char)

    out = []

    # 3. Desapilar (Orden Inverso - LIFO)
    while not stack.is_empty():
        out.append(stack.pop())

    # Reconstruir cadena final
    return ''.join(out)
```

Lógica LIFO

La propiedad fundamental **Last In, First Out** invierte secuencias naturalmente.

`O(n) Tiempo` `O(n) Espacio`

Extra: Notación Polaca

Las pilas son esenciales para convertir expresiones matemáticas infijas (A+B) a postfijas AB+ (Notación Polaca Inversa), usada por compiladores para evaluar sin paréntesis.

FLUJO DE DATOS

"ABC" → CBA (PILA) → "CBA"

Errores típicos

Fallos comunes en la implementación y uso de pilas estáticas

Ignorar Límites (Limits)

Olvidar verificar `is_full()` antes de un *push* (causa Overflow) o `is_empty()` antes de un *pop/peek* (causa Underflow).

Confusión Índice vs. Tamaño

Usar `top` como la cantidad de elementos. Recuerda: si `top = 0`, el tamaño es 1.

`size = top + 1`

Desincronización

Modificar `top` manualmente sin actualizar el array `_data`, rompiendo la coherencia del estado interno.

Referencias Fantasma (Memory Leak)

No limpiar la referencia en el array al hacer *pop*. El objeto sigue en memoria aunque `top` decremente.

EJEMPLO DE POP INCORRECTO:

```
def bad_pop(self):
  # ERROR: No limpia _data[top]
  item = self._data[self.top]
  self.top -= 1 # Top baja, pero item sigue ahí
  return item
```

⚠ Resultado: `_data` retiene objetos antiguos innecesariamente.

¿Qué devuelve size() en una pila vacía?

A 0

B -1

C None

D capacity

¿Qué devuelve size() en una pila vacía?

A 0

B -1

C None

D capacity

¿Cuál operación debe verificar underflow?

A push

B pop

C peek

D size

¿Cuál operación debe verificar underflow?

A push

B pop

C peek

D size

¿Qué indica top = capacity - 1?

A Vacía

B Llena

C Error

D Mitad

¿Qué indica top = capacity - 1?

A Vacía

B Llena

C Error

D Mitad

Resumen

Consolidación de conocimientos

Conceptos Clave

- **LIFO (Last In, First Out):** El último elemento en entrar es el primero en salir. Fundamental para call stacks y deshacer.
- **TAD vs Implementación:** El TAD define el comportamiento; la implementación define la estructura (array, lista).
- **Eficiencia O(1):** Las operaciones push, pop, peek deben ser constantes en tiempo.